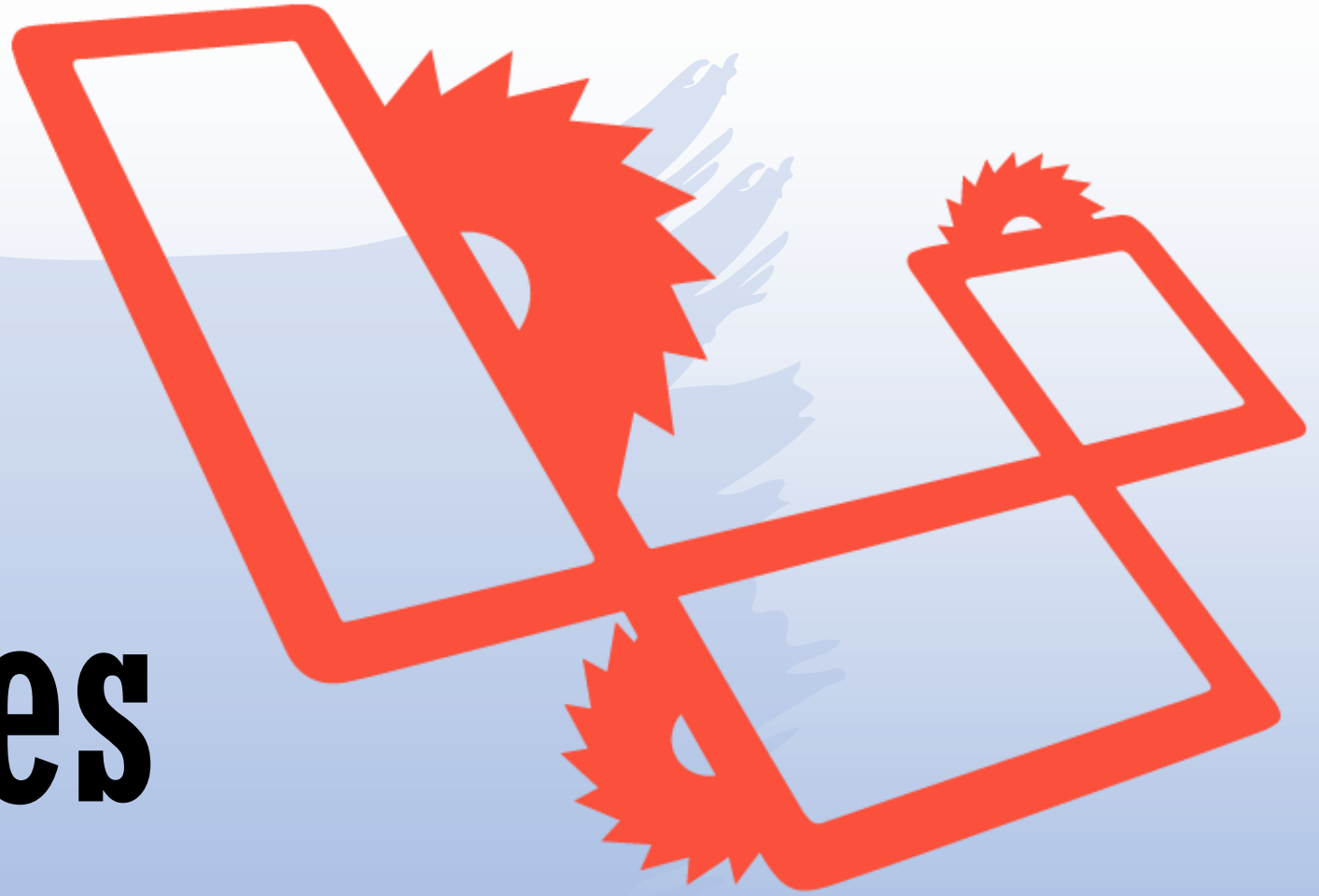


Les modèles Blade



Introduction

Blade est le moteur de modélisation simple mais puissant inclus avec Laravel. Contrairement à certains moteurs de modèles PHP, Blade ne vous empêche pas d'utiliser du code PHP simple dans vos modèles. En fait, tous les modèles Blade sont compilés en code PHP simple et mis en cache jusqu'à ce qu'ils soient modifiés, ce qui signifie que Blade n'ajoute pratiquement aucune surcharge à votre application. Les fichiers de modèle Blade utilisent l'extension de fichier **.blade.php** et sont généralement stockés dans le répertoire **resources/views**.

Introduction

Les vues Blade peuvent être renvoyées par des routes ou des contrôleurs à l'aide de la fonction **View**. Les données peuvent être transmises à la vue Blade à l'aide du deuxième argument de l'assistant de vue :

```
Route::get('/', function () {  
    return view('greeting', ['name' => 'Jack']);  
});
```



*Affichage des
données*

Affichage des données

Vous pouvez afficher les données qui sont transmises à vos vues Blade en enveloppant la variable dans des accolades. Par exemple, avec la route suivante :

```
Route::get('/', function () {  
    return view('welcome', ['name' => 'Samantha']);  
});
```

Vous pouvez afficher le contenu de la variable nom comme suit :

```
Hello, {{ $name }}.
```

Affichage des données

Vous n'êtes pas limité à l'affichage du contenu des variables passées à la vue. Vous pouvez également afficher les résultats de n'importe quelle fonction PHP. En fait, vous pouvez placer tout le code PHP que vous souhaitez à l'intérieur d'une instruction Blade echo :

```
The current UNIX timestamp is {{ time() }}.
```

Blade et les Framework JavaScript

Étant donné que de nombreux Framework JavaScript utilisent également des accolades pour indiquer qu'une expression donnée doit être affichée dans le navigateur, vous pouvez utiliser le symbole @ pour informer le moteur de rendu Blade qu'une expression ne doit pas être modifiée. Par exemple :

```
<h1>Laravel</h1>
```

```
Hello, @{{ name }}.
```

Blade et les Framework JavaScript

Dans l'exemple précédant, le symbole @ sera supprimé par Blade ; cependant, l'expression {{nom }} ne sera pas modifiée par le moteur Blade, ce qui lui permettra d'être rendue par votre framework JavaScript.

Le symbole @ peut également être utilisé pour échapper aux directives de Blade :

```
{{-- Blade template --}}
```

```
@@if()
```

```
<!-- HTML output -->
```

```
@if()
```


La directive @verbatim

Si vous affichez des **variables JavaScript** dans une grande partie de votre modèle, vous pouvez envelopper le HTML dans la directive **@verbatim** afin de ne pas devoir faire précéder chaque instruction Blade echo du symbole @ :

```
@verbatim
    <div class="container">
        Hello, {{ name }}.
    </div>
@endverbatim
```


*Directives de
Blade*

La directive @If

Vous pouvez construire des instructions if en utilisant les directives **@if**, **@elseif**, **@else**, et **@endif**. Ces directives fonctionnent de la même manière que leurs équivalents en PHP :

```
@if (count($records) == 1)
    I have one record!
}elseif (count($records) > 1)
    I have multiple records!
@else
    I don't have any records!
@endif
```

La directive @unless

Pour plus de commodité, Blade fournit également une directive **@unless** :

```
@unless (Auth::check())  
You are not signed in.  
@endunless
```

La directive @isset

En plus des directives conditionnelles déjà abordées, les directives **@isset** et **@empty** peuvent être utilisées comme raccourcis pratiques pour leurs fonctions PHP respectives :

```
@isset($records)  
// $records is defined and is not null...  
@endisset
```

```
@empty($records)  
// $records is "empty"...  
@endempty
```

Directives d'authentification

Les directives **@auth** et **@guest** peuvent être utilisées pour déterminer rapidement si l'utilisateur actuel est authentifié ou est un invité :

@auth

// The user is authenticated...

@endauth

@guest

// The user is not authenticated...

@endguest

Directives d'authentification

Si nécessaire, vous pouvez spécifier la garde d'authentification qui doit être vérifiée lors de l'utilisation des directives **@auth** et **@guest** :

```
@auth('admin')
```

```
// The user is authenticated...
```

```
@endauth
```

```
@guest('admin')
```

```
// The user is not authenticated...
```

```
@endguest
```

Directives d'environnement

Vous pouvez vérifier si l'application fonctionne dans l'environnement de production en utilisant la directive **@production** :

```
@production  
// Production specific content...  
@endproduction
```


Directives d'environnement

Vous pouvez également déterminer si l'application s'exécute dans un environnement spécifique à l'aide de la directive **@env** :

```
@env('staging')  
// The application is running in "staging"..  
@endenv
```

```
@env(['staging', 'production'])  
// The application is running in "staging" or "production"..  
@endenv
```

Directives de la section

Vous pouvez déterminer si une section d'héritage de modèle a du contenu en utilisant la directive **@hasSection** :

```
@hasSection('navigation')
  <div class="pull-right">
    @yield('navigation')
  </div>

  <div class="clearfix"></div>
@endif
```

Directives de la section

Vous pouvez utiliser la directive **sectionMissing** pour déterminer si une section n'a pas de contenu :

```
@sectionMissing('navigation')  
  <div class="pull-right">  
    @include('default-navigation')  
  </div>  
@endif
```

Directive Switch

Les instructions **switch** peuvent être construites en utilisant les directives **@switch**, **@case**, **@break**, **@default** et **@endswitch** :

```
@switch($i)
  @case(1)
    First case...
  @break
  .....
  @default
    Default case...
@endswitch
```

Les boucles

Blade fournit des directives simples pour travailler avec les structures de boucles de PHP. Encore une fois, chacune de ces directives fonctionne de manière identique à leurs homologues PHP :

```
@for ($i = 0; $i < 10; $i++)  
    The current value is {{ $i }}  
@endfor  
  
@foreach ($users as $user)  
    <p>This is user {{ $user->id }}</p>  
@endforeach
```

Les boucles

```
@forelse ($users as $user)
    <li>{{ $user->name }}</li>
@empty
    <p>No users</p>
@endforelse

@while (true)
    <p>I'm looping forever.</p>
@endwhile
```

Les boucles

Lorsque vous utilisez des boucles, vous pouvez également sauter l'itération en cours ou terminer la boucle à l'aide des directives **@continue** et **@break** :

```
@foreach ($users as $user)
    @if ($user->type == 1)
        @continue
    @endif
    <li>{{ $user->name }}</li>
    @if ($user->number == 5)
        @break
    @endif
@endforeach
```

Les boucles

Vous pouvez également inclure la condition de continuation ou de rupture dans la déclaration de la directive :

```
@foreach ($users as $user)
    @continue($user->type == 1)

    <li>{{ $user->name }}</li>

    @break($user->number == 5)
@endforeach
```


La variable Loop

Lors de l'itération dans une boucle foreach, une variable **\$loop** sera disponible à l'intérieur de votre boucle. Cette variable permet d'accéder à certaines informations utiles, telles que l'index actuel de la boucle et le fait qu'il s'agisse de la première ou de la dernière itération de la boucle :

```
@foreach ($users as $user)
    @if ($loop->first)
        Il s'agit de la première itération.
    @endif

    <p>This is user {{ $user->id }}</p>
@endforeach
```

La variable Loop

Si vous vous trouvez dans une boucle imbriquée, vous pouvez accéder à la variable **\$loop** de la boucle mère via la propriété **parent** :

```
@foreach ($users as $user)
    @foreach ($user->posts as $post)
        @if ($loop->parent->first)
            This is the first iteration of the parent loop.
        @endif
    @endforeach
@endforeach
```

La variable Loop

Propriété	Description
\$loop->index	L'index de l'itération de la boucle actuelle (commence à 0).
\$loop->iteration	L'itération actuelle de la boucle (commence à 1).
\$loop->remaining	Les itérations restantes dans la boucle.
\$loop->count	Le nombre total d'éléments dans le tableau en cours d'itération.
\$loop->first	Si c'est la première itération de la boucle.
\$loop->last	Si c'est la dernière itération de la boucle.
\$loop->even	Si c'est une itération paire dans la boucle.
\$loop->odd	Si c'est une itération impaire dans la boucle.
\$loop->depth	Le niveau d'imbrication de la boucle actuelle.
\$loop->parent	Dans une boucle imbriquée, la variable de boucle du parent.

Classes conditionnelles

La directive **@class** compile de manière conditionnelle une chaîne de classes CSS.

```
@php
$isActive = false;
$hasError = true;
@endphp
<span @class([
'font-bold' => $isActive,
'bg-red' => $hasError,
])></span>

<span class="bg-red"></span>
```

Attributs supplémentaires

Par commodité, vous pouvez utiliser la directive **@checked** pour indiquer facilement si une case à cocher HTML donnée est "cochée". Cette directive donnera un écho à la case cochée si la condition fournie est évaluée à **true** :

```
<input type="checkbox"  
name="active"  
value="active"  
@checked(old('active', $user->active)) />
```

Attributs supplémentaires

De même, la directive **@selected** peut être utilisée pour indiquer si une option de sélection donnée doit être "sélectionnée" :

```
<select name="version">
  @foreach ($product->versions as $version)
    <option value="{{ $version }}" @selected(old('version') == $version)>
      {{ $version }}
    </option>
  @endforeach
</select>
```

Attributs supplémentaires

En outre, la directive **@disabled** peut être utilisée pour indiquer si un élément donné doit être "désactivé" :

```
<button type="submit" @disabled($errors-  
>isEmpty())>Submit</button>
```

Attributs supplémentaires

En outre, la directive **@readonly** peut être utilisée pour indiquer si un élément donné doit être "readonly" :

```
<input type="email"  
      name="email"  
      value="email@laravel.com"  
      @readonly($user->isNotAdmin()) />
```


Inclure des sous-vues

La directive **@include** de Blade vous permet d'inclure une vue Blade à l'intérieur d'une autre vue. Toutes les variables qui sont disponibles pour la vue parente seront disponibles pour la vue incluse :

```
<div>
    @include('shared.errors')

    <form>
        <!-- Form Contents -->
    </form>
</div>
```

Inclure des sous-vues

Même si la vue incluse hérite de toutes les données disponibles dans la vue parent, vous pouvez également transmettre un tableau de données supplémentaires qui doivent être mises à la disposition de la vue incluse :

```
@include('view.name', ['status' => 'complete'])
```

Inclure des sous-vues

Si vous tentez d'**@inclure** une vue qui n'existe pas, Laravel émettra une erreur. Si vous souhaitez inclure une vue qui peut ou non être présente, vous devez utiliser la directive **@includelf** :

```
@includelf('view.name', ['status' => 'complete'])
```

Inclure des sous-vues

Si vous souhaitez **@inclure** une vue si une expression booléenne donnée donne la valeur vrai ou faux, vous pouvez utiliser les directives **@includeWhen** et **@includeUnless** :

```
@includeWhen($boolean, 'view.name', ['status' => 'complete'])
```

```
@includeUnless($boolean, 'view.name', ['status' => 'complete'])
```

Inclure des sous-vues

Pour inclure la première vue qui existe dans un tableau de vues donné, vous pouvez utiliser la directive **includeFirst** :

```
@includeFirst(['custom.admin', 'admin'], ['status' => 'complete'])
```

Rendu des vues pour les collections

Vous pouvez combiner des boucles et des inclusions en une seule ligne avec la directive **@each** de Blade :

```
@each('view.name', $jobs, 'job')
```

Le premier argument est la vue à rendre.

Le deuxième argument est le tableau ou la collection sur lequel vous souhaitez effectuer une itération.

Le troisième argument est le nom de la variable qui sera affectée à l'itération actuelle dans la vue.

Rendu des vues pour les collections

Vous pouvez également passer un quatrième argument à la directive **@each**.

Cet argument détermine la vue qui sera rendue si le tableau donné est vide.

```
@each('view.name', $jobs, 'job', 'view.empty')
```

PHP brut

Dans certaines situations, il est utile d'intégrer du code PHP dans vos vues. Vous pouvez utiliser la directive `@php` de Blade pour exécuter un bloc de code PHP simple dans votre modèle :

```
@php  
    $counter = 1;  
@endphp
```


PHP brut

Si vous n'avez besoin d'écrire qu'une seule instruction PHP, vous pouvez inclure cette instruction dans la directive **@php** :

```
@php($counter = 1)
```

Commentaires

Blade vous permet également de définir des commentaires dans vos vues. Cependant, contrairement aux commentaires HTML, les commentaires Blade ne sont pas inclus dans le HTML renvoyé par votre application :

```
{{-- Ce commentaire ne sera pas présent dans le HTML rendu. --}}
```

A blue brushstroke background shape, resembling a paint splatter or a hand-drawn oval, with irregular, feathered edges. It is centered on a white background.

Les Composants (Components)

Components

Les composants et les emplacements (slots) offrent les mêmes avantages que les sections, les mises en page et les inclusions ; toutefois, certains trouveront le modèle mental des composants et des emplacements plus facile à comprendre. Il existe deux approches pour écrire des composants :

- Les composants basés sur des classes
- Les composants anonymes.

Components

Pour créer un composant basé sur une classe, vous pouvez utiliser la commande **Artisan make:component**. Pour illustrer l'utilisation des composants, nous allons créer un simple composant **Alert**. La commande **make:component** placera le composant dans le répertoire **app/View/Components** :

```
php artisan make:component Alert
```

Components

La commande **make:component** créera également un modèle de vue pour le composant. La vue sera placée dans le répertoire **resources/views/components**. Lorsque vous écrivez des composants pour votre propre application, les composants sont automatiquement découverts dans le répertoire **app/View/Components** et le répertoire **resources/views/components**, de sorte qu'aucun autre enregistrement de composant n'est généralement nécessaire.

Components

Vous pouvez également créer des composants dans des sous-répertoires :

```
php artisan make:component Forms/Input
```

La commande ci-dessus va créer un composant Input dans le répertoire **app/View/Components/Forms** et la vue sera placée dans le répertoire **resources/views/components/forms**.

composant anonyme

Si vous souhaitez créer un composant anonyme (un composant avec seulement un modèle Blade et aucune classe), vous pouvez utiliser le drapeau **--view** lorsque vous invoquez la commande **make:component** :

```
php artisan make:component forms.input --view
```

La commande ci-dessus créera un fichier Blade à l'adresse **resources/views/components/forms/input.blade.php** qui pourra être rendu comme un composant via **<x-forms.input />**.

Enregistrement manuel des composants du paquet

Lorsque vous écrivez des composants pour votre propre application, les composants sont automatiquement découverts dans le répertoire **app/View/Components** et le **répertoire resources/views/components**.

Cependant, si vous construisez un paquet qui utilise des composants Blade, vous devrez enregistrer manuellement votre classe de composant et son alias de balise HTML. Vous devez généralement enregistrer vos composants dans la méthode **boot** du fournisseur de services de votre paquet

```
public function boot()  
{  
    Blade::component('package-alert', Alert::class); }  
}
```

Enregistrement manuel des composants du paquet

Une fois que votre composant a été enregistré, il peut être rendu en utilisant son alias de balise :

```
<x-package-alert/>
```

Vous pouvez également utiliser la méthode **componentNamespace** pour charger automatiquement les classes de composants par convention.

Par exemple, un paquetage **Nightshade** peut comporter des composants **Calendar** et **ColorPicker** qui résident dans l'espace de noms **Package \ Views \ Components** :

Enregistrement manuel des composants du paquet

Une fois que votre composant a été enregistré, il peut être rendu en utilisant son alias de balise :

```
public function boot()  
{  
    Blade::componentNamespace('Nightshade\\Views\\Components',  
        'nightshade');  
}
```

Enregistrement manuel des composants du paquet

Cela permettra l'utilisation des composants de packaging par leur espace de nom de fournisseur en utilisant la syntaxe **package-name::** :

```
<x-nightshade::calendar />  
<x-nightshade::color-picker />
```

Blade détectera automatiquement la classe qui est liée à ce composant en mettant en majuscule le nom du composant.

Les sous-répertoires sont également pris en charge en utilisant la notation "point".

Rendu des composants

Pour afficher un composant, vous pouvez utiliser une balise de composant Blade dans l'un de vos modèles Blade.

Les balises de composant Blade commencent par la chaîne **x-** suivie du nom de la classe de composant en casse kebab :

```
<x-alert/>
```

```
<x-user-profile/>
```

Rendu des composants

Si la classe du composant est imbriquée plus profondément dans le répertoire **app/View/Components**, vous pouvez utiliser le caractère `.` pour indiquer l'imbrication des répertoires. Par exemple, si nous supposons qu'un composant se trouve dans le répertoire **app/View/Components/Inputs/Button.php**, nous pouvons le rendre de la manière suivante :

```
<x-inputs.button/>
```

Transmettre des données aux composants

Vous pouvez transmettre des données aux composants Blade en utilisant des attributs HTML. Les valeurs primitives codées en dur peuvent être transmises au composant à l'aide de simples chaînes d'attributs HTML. Les expressions et les variables PHP doivent être transmises au composant par le biais d'attributs qui utilisent le caractère `:` comme préfixe :

```
<x-alert type="error" :message="$message"/>
```

Transmettre des données aux composants

Vous devez définir tous les attributs de données du composant dans le constructeur de sa classe.

Toutes les propriétés publiques d'un composant seront automatiquement mises à la disposition de la vue du composant. Il n'est pas nécessaire de transmettre les données à la vue à partir de la méthode de rendu du composant :


```
<?php
```

```
namespace App\View\Components;
```

```
use Illuminate\View\Component;
```

```
class Alert extends Component
```

```
{
```

```
    public $type;
```

```
    public $message;
```

```
    public function __construct($type, $message)
```

```
    {
```

```
        $this->type = $type;
```

```
        $this->message = $message;
```

```
    }
```

```
    public function render()
```

```
    {
```

```
        return view('components.alert');
```

```
    }
```

```
}
```

Transmettre des données aux composants

Lorsque votre composant est rendu, vous pouvez afficher le contenu des variables publiques de votre composant en faisant écho aux variables par leur nom :

```
<div class="alert alert-{{ $type }}">
    {{ $message }}
</div>
```

Transmettre des données aux composants

Les arguments des constructeurs de composants doivent être spécifiés en utilisant la casse **camel**, tandis que la casse **kebab** doit être utilisée pour faire référence aux noms des arguments dans vos attributs HTML.

Par exemple, avec le constructeur de composant suivant :

```
public function __construct($alertType)
{
    $this->alertType = $alertType;
}
```

L'argument **\$alertType** peut être fourni au composant de la manière suivante :

```
<x-alert alert-type="danger" />
```

Transmettre des données aux composants

Syntaxe courte des attributs

Lorsque vous transmettez des attributs aux composants, vous pouvez également utiliser une syntaxe "**attribut court**". C'est souvent pratique car les noms des attributs correspondent souvent aux noms des variables auxquelles ils correspondent :

```
{{-- Short attribute syntax... --}}
```

```
<x-profile :$userId :$name />
```

```
{{-- Is equivalent to... --}}
```

```
<x-profile :user-id="$userId" :name="$name" />
```

Transmettre des données aux composants

Méthodes des composants

Tous les variables publiques sont disponibles pour votre modèle de composant et toutes les méthodes publiques du composant peuvent être invoquées.

Par exemple, imaginez un composant qui possède une méthode **isSelected** :

```
public function isSelected($option)
{
    return $option === $this->selected;
}
```

Transmettre des données aux composants

Méthodes des composants

Vous pouvez exécuter cette méthode à partir de votre modèle de composant en invoquant la variable correspondant au nom de la méthode :

```
<option {{ $isSelected($value) ? 'selected' : '' }} value="{{ $value }}">
    {{ $label }}
</option>
```

Attributs des composants

En général, si vous souhaitez transmettre des attributs HTML supplémentaires à l'élément racine du modèle de composant.

Par exemple, imaginons que nous voulions rendre un composant d'alerte comme suit :

```
<x-alert type="error" :message="$message" class="mt-4"/>
```

Attributs des composants

Tous les attributs qui ne font pas partie du constructeur du composant seront automatiquement ajoutés au "sac d'attributs" du composant. Ce sac d'attributs est automatiquement mis à la disposition du composant via la variable **\$attributes**. Tous les attributs peuvent être rendus dans le composant en utilisant cette variable :

```
<div {{ $attributes }}>  
<!-- Component content -->  
</div>
```


Attributs des composants

Attributs par défaut / fusionnés

Parfois, vous pouvez avoir besoin de spécifier des valeurs par défaut pour les attributs ou de fusionner des valeurs supplémentaires dans certains attributs du composant. Pour ce faire, vous pouvez utiliser la méthode de fusion du sac d'attributs. Cette méthode est particulièrement utile pour définir un ensemble de classes CSS par défaut qui doivent toujours être appliquées à un composant :

```
<div {{ $attributes->merge(['class' => 'alert alert-'. $type]) }}>
    {{ $message }}
</div>
```

Attributs des composants

Attributs par défaut / fusionnés

Si nous supposons que ce composant est utilisé de la manière suivante :

```
<x-alert type="error" :message="$message" class="mb-4"/>
```

Le HTML final, rendu, du composant ressemblera à ce qui suit :

```
<div class="alert alert-error mb-4">  
  <!-- Contenu de la variable $message -->  
</div>
```


Attributs des composants

Fusionner des classes sous conditions

Si vous devez fusionner d'autres attributs avec votre composant, vous pouvez enchaîner la méthode de fusion avec la méthode de classe :

```
<button {{ $attributes->class(['p-4'])->merge(['type' => 'button']) }}>
    {{ $slot }}
</button>
```

Slots

Vous aurez souvent besoin de transmettre du contenu supplémentaire à votre composant via des "**slots**". Les slots des composants sont rendus par l'affichage de la variable **\$slot**. Pour explorer ce concept, imaginons qu'un composant d'alerte possède le balisage suivant :

```
<div class="alert alert-danger">  
  {{ $slot }}  
</div>
```

Slots

Nous pouvons transmettre du contenu au slot en injectant du contenu dans le composant :

```
<x-alert>  
  <strong>Whoops!</strong> Quelque chose a mal tourné!  
</x-alert>
```

Slots

Il arrive parfois qu'un composant doive rendre plusieurs slots différents à différents endroits du composant. Modifions notre composant d'alerte pour permettre l'injection d'un emplacement "titre" :

```
<span class="alert-title">{{ $titre }}</span>
```

```
<div class="alert alert-danger">  
    {{ $slot }}  
</div>
```

Slots

Vous pouvez définir le contenu de l'emplacement nommé en utilisant la balise **x-slot**. Tout contenu ne figurant pas dans une balise **x-slot** explicite sera transmis au composant dans la variable **\$slot** :

```
<x-alert>
  <x-slot:titre>
    Erreur de serveur
  </x-slot>

  <strong>Whoops!</strong> Quelque chose a mal tourné!
</x-alert>
```


Vues des composants Inline

Pour les très petits composants, il peut être fastidieux de gérer à la fois la classe du composant et le modèle de vue du composant. Pour cette raison, vous pouvez retourner le balisage du composant directement à partir de la méthode de rendu :

```
public function render()
{
    return <<<'blade'
        <div class="alert alert-danger">
            {{ $slot }}
        </div>
        blade;
}
```

Vues des composants Inline

Pour créer un composant qui rend une vue Inline, vous pouvez utiliser l'option **--inline** lorsque vous exécutez la commande **make:component** :

```
php artisan make:component Alert --inline
```

Composants dynamiques

Il peut arriver que vous ayez besoin de rendre un composant mais que vous ne sachiez pas quel composant doit être rendu avant l'exécution.

Dans ce cas, vous pouvez utiliser le composant dynamique intégré de Laravel pour rendre le composant en fonction d'une valeur ou d'une variable d'exécution :

```
<x-dynamic-component :component="$componentName" class="mt-4" />
```



*Composants
anonymes*

Composants anonymes

Les composants anonymes fournissent un mécanisme de gestion d'un composant via un seul fichier. Cependant, les composants anonymes utilisent un seul fichier de vue et n'ont pas de classe associée.

Pour définir un composant anonyme, il vous suffit de placer un modèle Blade dans votre répertoire **resources/views/components**.

```
<x-alert/>
```

alert est le nom du composant (**alert.blade.php**)

Attributs / Propriétés des données

Vous pouvez utiliser le caractère . pour indiquer si un composant est imbriqué plus profondément dans le répertoire des composants.

Par exemple, si le composant est défini dans le répertoire **resources/views/components/inputs/button.blade.php**, vous pouvez le rendre de la manière suivante :

```
<x-inputs.button/>
```



*Création de
mises en page*

Définition du composant de mise en page

Par exemple, imaginons que nous construisions une application de liste de choses à faire. Nous pourrions définir un composant de mise en page qui ressemble à ce qui suit

```
<!-- resources/views/components/layout.blade.php -->
<html>
<head>
    <title>{{ $title ?? 'Todo Manager' }}</title>
</head>
<body>
    <h1>Todos</h1>
    <hr/>
    {{ $slot }}
</body></html>
```


Application du composant de mise en page

Une fois que le composant de mise en page a été défini, nous pouvons créer une vue Blade qui utilise le composant. Dans cet exemple, nous allons définir une vue simple qui affiche notre liste de tâches :

```
<!-- resources/views/tasks.blade.php -->
```

```
<x-layout>
```

```
    @foreach ($tasks as $task)
```

```
        {{ $task }}
```

```
    @endforeach
```

```
</x-layout>
```

Mises en page utilisant l'héritage des modèles

Les mises en page peuvent également être créées par "héritage de modèles". C'était la principale façon de construire des applications avant l'introduction des composants.

Pour commencer, examinons un exemple simple. Tout d'abord, nous allons examiner une mise en page. Étant donné que la plupart des applications Web conservent la même mise en page générale sur plusieurs pages, il est pratique de définir cette mise en page comme une seule vue Blade :

```
<!-- resources/views/layouts/app.blade.php -->
<html>
  <head>
    <title> Nom de l'application - @yield('title')</title>
  </head>
  <body>
    @section('sidebar')
      Il s'agit de la barre latérale principale.
    @show

    <div class="container">
      @yield('content')
    </div>
  </body>
</html>
```

Mises en page utilisant l'héritage des modèles

Comme vous pouvez le constater, ce fichier contient un balisage HTML typique. Cependant, il faut noter les directives **@section** et **@yield**.

La directive **@section**, comme son nom l'indique, définit une section de contenu, tandis que la directive **@yield** est utilisée pour afficher le contenu d'une section donnée.

Extension d'une mise en page

Lors de la définition d'une vue enfant, utilisez la directive Blade **@extends** pour préciser de quelle mise en page la vue enfant doit "hériter". Les vues qui étendent un modèle Blade peuvent injecter du contenu dans les sections du modèle en utilisant les directives **@section**. N'oubliez pas que, comme dans l'exemple ci-dessus, le contenu de ces sections sera affiché dans la mise en page à l'aide de **@yield** :

```
<!-- resources/views/child.blade.php -->
```

```
@extends('layouts.app')
```

```
@section('title', 'Page Title')
```

```
@section('sidebar')
```

```
@parent
```

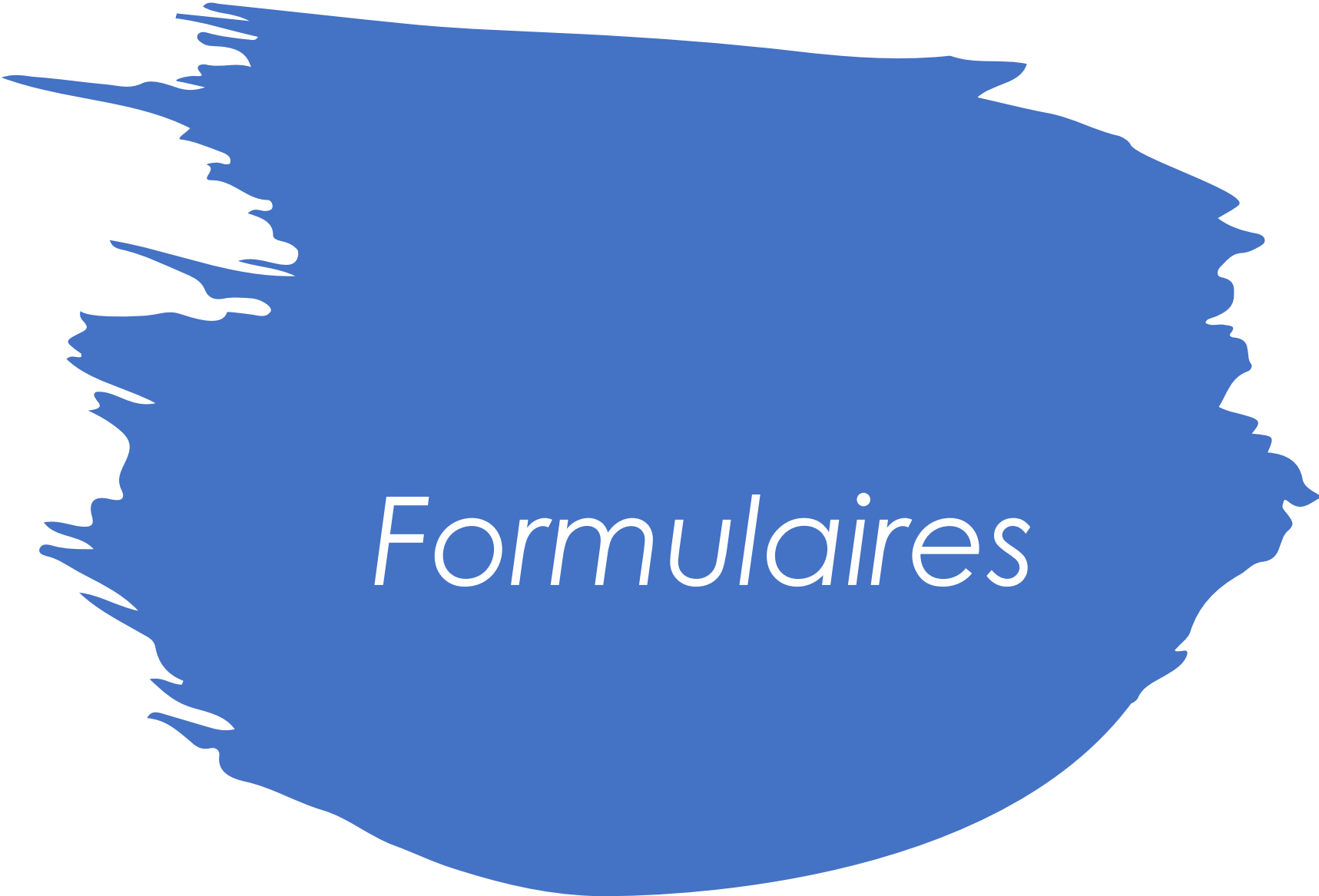
```
<p> Ceci est ajouté à la barre latérale principale.</p>
```

```
@endsection
```

```
@section('content')
```

```
<p> C'est le contenu de mon corps.</p>
```

```
@endsection
```



Formulaires

Le champ CSRF

Chaque fois que vous définissez un formulaire HTML dans votre application, vous devez inclure un champ de jeton **CSRF** caché dans le formulaire afin que le middleware de protection CSRF puisse valider la requête. Vous pouvez utiliser la directive Blade **@csrf** pour générer ce champ :

```
<form method="POST" action="/profile">  
  @csrf  
  
  ...  
</form>
```


Champ de la méthode HTTP

Comme les formulaires HTML ne peuvent pas effectuer de requêtes **PUT**, **PATCH** ou **DELETE**, vous devrez ajouter un champ **_method** caché pour utiliser ces verbes HTTP.

La directive Blade **@method** peut créer ce champ pour vous :

```
<form action="/foo/bar" method="POST">  
  @method('PUT')  
  
  ...  
</form>
```

Erreurs de validation

La directive **@error** peut être utilisée pour vérifier rapidement si des messages d'erreur de validation existent pour un attribut donné. Dans une directive @error, vous pouvez faire afficher la variable **\$message** pour afficher le message d'erreur :

```
<input id="title"
      type="text"
      class="@error('title') is-invalid @enderror">

@error('title')
    <div class="alert alert-danger">{{ $message }}</div>
@enderror
```

Erreurs de validation

Comme la directive **@error** se compile en une instruction "if", vous pouvez utiliser la directive **@else** pour rendre le contenu lorsqu'il n'y a pas d'erreur pour un attribut :

```
<label for="email">Email address</label>
```

```
<input id="email"  
  type="email"  
  class="@error('email') is-invalid @else is-valid @enderror">
```

A large, horizontal, blue brushstroke shape with irregular, feathered edges. The word "Piles" is written in the center in a white, italicized serif font.

Piles

Piles nommées

Blade vous permet de pousser vers des piles nommées qui peuvent être rendues ailleurs dans une autre vue ou disposition. Cela peut être particulièrement utile pour spécifier toutes les bibliothèques JavaScript requises par vos vues enfant :

```
@push('scripts')  
  <script src="/example.js"></script>  
@endpush
```

Piles nommées

Si vous souhaitez **@push** le contenu si une expression booléenne donnée est évaluée comme vraie, vous pouvez utiliser la directive **@pushIf** :

```
@pushIf($shouldPush, 'scripts')  
  <script src="/example.js"></script>  
@endPushIf
```

Piles nommées

Vous pouvez pousser sur une pile autant de fois que nécessaire. Pour afficher le contenu complet de la pile, passez le nom de la pile à la directive **@stack** :

```
<head>
  <!-- Head Contents -->

  @stack('scripts')
</head>
```

Piles nommées

Si vous souhaitez ajouter du contenu au début d'une pile, vous devez utiliser la directive **@prepend** :

```
@prepend('scripts')  
  <script src="/example.js"></script>  
@endprepend
```




*Injection de
service*

Injection de service

La directive **@inject** peut être utilisée pour récupérer un service dans le conteneur de services Laravel. Le premier argument passé à **@inject** est le nom de la variable dans laquelle le service sera placé, tandis que le second argument est le nom de la classe ou de l'interface du service que vous souhaitez résoudre :

```
@inject('metrics', 'App\Services\MetricsService')
```

```
<div>
```

```
    Monthly Revenue: {{ $metrics->monthlyRevenue() }}.
```

```
</div>
```

A blue brushstroke graphic with a rough, torn edge on the left side, set against a white background.

*Rendu des
modèles de
Blade en ligne*

Rendu des modèles de Blade inline

Parfois, vous pouvez avoir besoin de transformer une chaîne brute de modèle Blade en HTML valide. Vous pouvez accomplir cela en utilisant la méthode **render** fournie par la façade Blade. La méthode **render** accepte la chaîne du modèle Blade et un tableau optionnel de données à fournir au modèle :

```
use Illuminate\Support\Facades\Blade;  
  
return Blade::render('Hello, {{ $name }}', ['name' => 'ADDY Boubaker']);
```

Rendu des modèles de Blade inline

Laravel rend les modèles Blade inline en les écrivant dans le répertoire **storage/framework/views**. Si vous souhaitez que Laravel supprime ces fichiers temporaires après le rendu du modèle Blade, vous pouvez fournir l'argument **deleteCachedView** à la méthode :

```
return Blade::render(  
    'Hello, {{ $name }}',  
    ['name' => 'Julian Bashir'],  
    deleteCachedView: true  
);
```